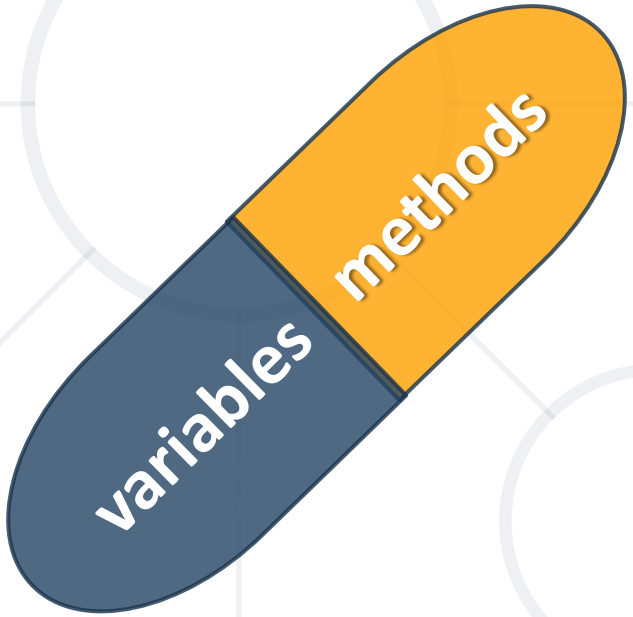
Encapsulation

Benefits of Encapsulation



SoftUni Team

Technical Trainers

Software University

SoftUni Foundation

Software University

http://softuni.bg

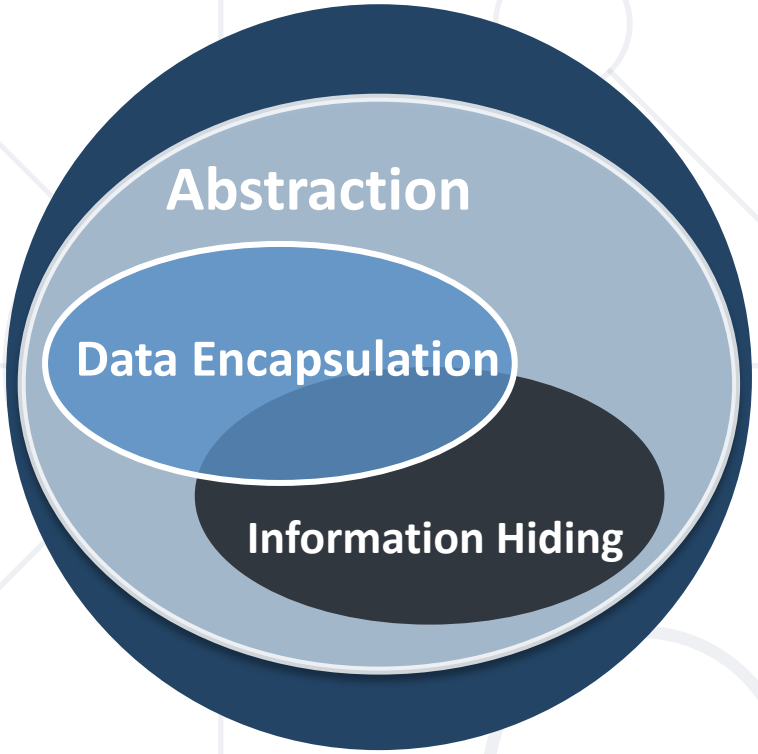
Table of Contents

Questions

SoftUni Foundation

sli.do

#fund-csharp



Hiding Implementation

Encapsulation

Encapsulation

- Process of wrapping code and data together into a single unit
- Flexibility and extensibility of the code
- Reduces **complexity**
- Structural changes remain **local**
- Allows **validation** and **data binding**

Encapsulation – Example

- Fields should be **private**

```
Person
-name: string
-age: int
+Person(string name, int age)
+Name: string
+Age: int
```

- == private

+ == public

- Properties should be **public**

Keyword this

- Reference to the **current object**
- Refers to the **current instance** of the class
- Can be passed as a **parameter to other methods**
- Can be **returned** from method
- Can invoke **current class methods**

Access Modifiers

Visibility of Class Members

Private Access Modifier

- Main way to perform encapsulation and hide data from the outside world

```
private string name;
Person (string name)
{
  this.name = name;
}
```

- **private** is the default field and method modifier
- Avoid declaring private classes and interfaces
  - Accessible only within the declared class itself

Public Access Modifier

- The most **permissive** access level
- There are **no restrictions** on accessing public members

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

- To access class directly from a namespace you can use the **using** keyword to include the namespace

Internal Access Modifier

- **internal** is the default class access modifier

```
class Person
{
    internal string Name { get; set; }
    internal int Age { get; set; }
}
```

- Accessible to any other class in the same project

```
Team rm = new Team("Real");
rm.Name = "Real Madrid";
```

Problem: Sort Persons by Name and Age

- Create a class Person

Person
-firstName:string -lastName:string -age:int
+FirstName():string +Age():int +toString():string

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Sort Persons by Name and Age (2)

SoftUni Foundation

```
public class Person
{
   private string firstName;
   private string lastName;
   private int age;
   public string FirstName => this.firstName;
   public string LastName => this.lastName;
   public int Age => this.age;
   public override string ToString() {
      return $"{FirstName} {LastName} is {Age} years old.";
   }
}
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Sort Persons by Name and Age

SoftUni Foundation

```
var lines = int.parse(Console.ReadLine());
var people = new List<Person>();
for (int i = 0; i < lines; i++;) {
  var cmdArgs = Console.ReadLine().Split();
  var person = new Person();
  person.FirstName = cmdArgs[0];
  //TODO: Set LastName and Age
  people.Add(person);
}
var sorted = people.OrderBy(p => p.FirstName)
        .ThenBy(p => p.Age).ToList();
//TODO: Print each person
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Problem: Salary Increase

- Expand Person with **salary**
- Add **getter** for **salary**
- Add a **method**, which updates salary with a given percent
- Persons younger than 30 get **half of the normal** increase

Person
-firstName: string -lastName: string -age: int -salary: double
+FirstName: string +Age: int +Salary: double +IncreaseSalary(int): void +ToString(): string

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Salary Increase

```
private double salary;
public double Salary {
  get => this.salary;
  private set => this.salary = value;
}
public void IncreaseSalary(double percent)
{
   if (this.age > 30)
     this.Salary += this.salary * percent / 100;
   else
     this.Salary += this.salary * percent / 200;
}
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Validation

Validation

- Setters are good place for simple **data validation**

```
public double Salary
{
  set
  {
    if (value < 460)
      throw new ArgumentException("...");
    this.salary = value;
  }
}
```

Throw **exceptions**, rather than printing to the Console

- Callers of your methods should take care of handling exceptions

Validation (2)

- Constructors use **private setters** with validation logic

```
public Person(string firstName, string lastName,
              int age, double salary)
{
  this.FirstName = firstName;
  this.LastName = lastName;
  this.Age = age;
  this.Salary = salary;
}
```

Validation happens inside the setter

- Guarantee **valid state** of the object after its creation

Problem: Validate Data

- Expand Person with **validation for every field**
- Names must be **at least 3 symbols**
- Age **cannot be zero or negative**
- Salary **cannot be less than 460**

Person
-firstName: string -lastName: string -age: int -salary: double
+Person() +FirstName(string fname) +LastName(string lname) +Age(int age) +Salary(double salary)

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Validate Data

SoftUni Foundation

```
public int Age {
   get => this.age;
   private set {
      if (age < 1)
         throw new ArgumentException("...");
      this.age = value;
   }
}
//TODO: Add validation for firstName
//TODO: Add validation for lastName
//TODO: Add validation for salary
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Mutable vs Immutable Objects

SoftUni Foundation

- Mutable Objects
  - The contents of that instance **can** be altered

```
Point myPoint = new Point( 0.0, 0.0 );
Console.WriteLine( myPoint );
myPoint.SetLocation( 1.0, 0.0 );
Console.WriteLine( myPoint );
```

```
0.0, 0.0
1.0, 0.0
```

- Immutable Objects
  - The contents of the instance **can't** be altered

```
string myString = "old String"
Console.WriteLine(myString);
myString.replaceAll("old", "new");
Console.WriteLine( myString );
```

```
old String
old String
```

Mutable Fields

- **Private mutable** fields are still **not encapsulated**

```
class Team
{
    private List<Person> players;
    public List<Person> Players
    {
        get { return this.players; }
    }
}
```

- In this case you can **access** the field methods **through the getter**

Mutable Fields

- You can use **IReadOnlyCollection** to encapsulate collections

```
public class Team
{
  private List<Person> players;
  public IReadOnlyCollection<Person> Players
  {
    get { return this.players.AsReadOnly(); }
  }
  public void AddPlayer(Person player)
  { this.players.Add(player); }
}
```

Problem: Team

- Team have two squads
  - first team & reserve team
- Read persons from console and add them to team
- If they are younger than 40, they go to first squad
- Print both squad sizes

Team
-name : string -firstTeam: List<Person> -reserveTeam: List<Person>
+Team(String name) +Name(): string +FirstTeam(): ReadOnlyList<Person> +ReserveTeam: ReadOnlyList<Person> +addPlayer(Person person)

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Team

```
private string name;
private List<Person> firstTeam;
private List<Person> reserveTeam;

public Team(string name)
{
    this.name = name;
    this.firstTeam = new List<Person>();
    this.reserveTeam = new List<Person>();
}
//continues on the next slide
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Solution: Team(2)

```
public IReadOnlyCollection<Person> FirstTeam
{
    get { return this.firstTeam.AsReadOnly(); }
}
//TODO: Implement reserve team getter
public void AddPlayer(Person player)
{
    if (player.Age < 40)
      firstTeam.Add(player);
    else
      reserveTeam.Add(player);
}
```

Check your solution here: https://judge.softuni.bg/Contests/678/Encapsulation-Lab

Summary

- Encapsulation:
  - Hides **implementation**
  - Reduces **complexity**
  - Ensures that structural changes remain local
- **Mutable** and **Immutable** objects

Questions?

SoftUni

Software University

SoftUni Svetlina

SoftUni Creative

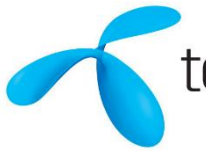
SoftUni Digital

SoftUni Foundation

SoftUni Kids

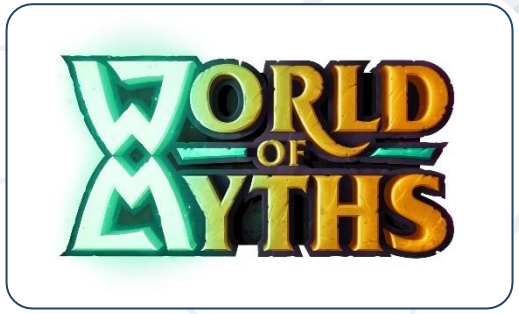
SoftUni Diamond Partners

SoftUni Foundation



SoftUni Organizational Partners

SoftUni Foundation



Trainings @ Software University (SoftUni)

SoftUni Foundation

- Software University – High-Quality Education and Employment Opportunities
  - softuni.bg
- Software University Foundation
  - http://softuni.foundation/
- Software University @ Facebook
  - facebook.com/SoftwareUniversity
- Software University Forums
  - forum.softuni.bg

License

- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license